Méthode des éléments finis en 2D

Mattis Duhaa

1 Introduction

Ce document a pour but de décrire une méthode numérique de façon assez précise pour qu'une IA puisse l'implémenter en Python.

Soit Ω un rectangle et $\beta \subset \Omega$ un obstacle (défini à la section 3.1). On pose $D = \Omega \setminus \beta$, dont le bord est $\partial D = \partial\Omega \cup \partial\beta$. Pour $f \in L^2(D)$ et g_β une fonction régulière définie au voisinage de $\partial\beta$, données, on souhaite résoudre le problème

$$(P) \quad \begin{cases} -\Delta u + u = f & \text{dans } D, \\ u = 0 & \text{sur } \partial\Omega, \\ u = g_\beta & \text{sur } \partial\beta. \end{cases}$$

Le cas $g_\beta = 0$ correspond à la condition homogène $u = 0$ sur ∂D.

2 Formulation variationnelle

On pose $V = H_0^1(D)$ et

$$V_g = \{v \in H^1(D) \ : \ v = 0 \text{ sur } \partial\Omega, \quad v = g_\beta \text{ sur } \partial\beta\},$$

où les égalités au bord s'entendent au sens des traces. V_g est non vide dès que $g_\beta \in H^{1/2}(\partial\beta)$, ce qui est le cas pour g_β régulière : on peut construire un *relèvement* $u_g \in V_g$, par exemple $u_g = \chi\, g_\beta$ avec χ une fonction régulière égale à 1 près de β et nulle près de $\partial\Omega$ (possible car $\partial\beta$ et $\partial\Omega$ sont disjoints). On a alors $V_g = u_g + V$.

Soit $v \in V$. On multiplie l'équation par v et on intègre sur D. La formule de Green donne

$$\int_D -\Delta u\, v = \int_D \nabla u \cdot \nabla v - \underbrace{\int_{\partial D} \frac{\partial u}{\partial n}\, v \, \mathrm{d}s}_{=0 \text{ car } v=0 \text{ sur } \partial D},$$

d'où

$$\int_D -\Delta u\, v + \int_D u\, v = \int_D \nabla u \cdot \nabla v + \int_D u\, v.$$

Les fonctions test restent nulles sur tout ∂D : seule l'inconnue u porte la condition $u = g_\beta$. On définit

$$a(u, v) = \int_D \nabla u \cdot \nabla v + \int_D u\, v,$$
$$\ell(v) = \int_D f\, v.$$

Problème continu.

$$(P_V) \quad \text{Trouver } u \in V_g \text{ tel que } a(u, v) = \ell(v) \quad \forall v \in V.$$

En écrivant $u = u_g + u_0$ avec $u_0 \in V$, cela équivaut à

$$\text{Trouver } u_0 \in V \text{ tel que } a(u_0, v) = \ell(v) - a(u_g, v) \quad \forall v \in V,$$

qui est un problème homogène avec un second membre modifié.

Problème discret. On note W_h l'espace des fonctions continues, P_2 ou Q_2 par morceaux, sans condition au bord, et $V_h = W_h \cap \{v = 0 \text{ sur } \partial D_h\}$ (section 4.1). On remplace g_β par son interpolée aux nœuds de $\partial\beta$:

$$V_{g,h} = \{v_h \in W_h \ : v_h(x_j) = 0 \text{ pour les nœuds } x_j \in \partial\Omega,$$
$$v_h(x_j) = g_\beta(x_j) \text{ pour les nœuds } x_j \in \partial\beta\},$$

$$(P_h) \quad \text{Trouver } u_h \in V_{g,h} \text{ tel que } a(u_h, v_h) = \ell(v_h) \quad \forall v_h \in V_h.$$

Le relèvement discret est $u_{g,h} = \sum_{x_j \in \partial\beta} g_\beta(x_j)\, \varphi_j$ (fonctions de base nodales de W_h), et $V_{g,h} = u_{g,h} + V_h$. Les conditions au bord sont donc contenues dans la définition de $V_{g,h}$: elles fixent les valeurs aux nœuds de bord (section 5.6).

Théorème (Lax–Milgram)**.** *Soit V un espace de Hilbert, a une forme bilinéaire continue et coercive sur V et ℓ une forme linéaire continue sur V. Alors il existe un unique $u \in V$ tel que $a(u, v) = \ell(v)$ pour tout $v \in V$.*

Ici a est exactement le produit scalaire de $H^1(D)$. Par l'inégalité de Cauchy–Schwarz,

$$|a(u,v)| \le \|u\|_{H^1}\|v\|_{H^1}, \qquad a(v,v) = \|v\|^2_{H^1}, \qquad |\ell(v)| \le \|f\|_{L^2}\|v\|_{H^1}.$$

a est donc continue et coercive (de constante 1) et ℓ est continue, de même que $v \mapsto \ell(v) - a(u_g, v)$. Le théorème s'applique à $u_0 \in V$ et à $u_h - u_{g,h} \in V_h$: (P_V) et (P_h) ont chacun une unique solution.

Estimation d'erreur (lemme de Céa). En retranchant (P_h) à (P_V) avec $v = v_h$, on obtient $a(u - u_h, v_h) = 0$ pour tout $v_h \in V_h$. Soit $w_h \in V_{g,h}$: comme $u_h - w_h \in V_h$,

$$\|u-u_h\|^2_{H^1} = a(u-u_h, u-w_h) + a(u-u_h, w_h-u_h) = a(u-u_h, u-w_h) \le \|u-u_h\|_{H^1}\,\|u-w_h\|_{H^1},$$

donc

$$\|u - u_h\|_{H^1} = \min_{w_h \in V_{g,h}} \|u - w_h\|_{H^1}.$$

L'erreur ne dépend donc que de la capacité de $V_{g,h}$ à approcher u. L'interpolée nodale $I_h u$ appartient à $V_{g,h}$, puisque $u = g_\beta$ aux nœuds de $\partial\beta$. Pour des éléments de degré 2 et u assez régulière (ce qui demande g_β régulière), on attend $\|u - u_h\|_{H^1} = O(h^2)$ et $\|u - u_h\|_{L^2} = O(h^3)$.

Remarque. Avec des triangles courbes, le domaine maillé D_h n'est pas exactement D : en toute rigueur $V_h \not\subset H^1_0(D)$, et g_β n'est imposée exactement qu'aux nœuds de $\partial\beta$. L'erreur géométrique commise est d'un ordre supérieur et ne dégrade pas les taux ci-dessus.

3 Maillage en 2D

L'idée est de réaliser un maillage autour d'un obstacle β à bord courbe, éventuellement non convexe, mais étoilé par rapport à un point c (chaque demi-droite issue de c coupe $\partial\beta$ une seule fois). Le domaine $\Omega = [0, L_x] \times [0, L_y]$ est un rectangle, avec $L_x = n_x h$ et $L_y = n_y h$, que l'on recouvre d'une grille cartésienne de pas h. Le maillage couvre $\Omega \setminus \beta$, que l'on découpe en deux zones :

- une zone $\mathcal{Q}$, formée des cellules de la grille qui ne rencontrent pas β et restent à une distance au moins m_β de β ; ces cellules sont des quadrilatères (éléments Q_2) et recouvrent Ω jusqu'à son bord ;
- le reste $\mathcal{T} = (\Omega\setminus\beta)\setminus\mathcal{Q}$, une zone autour de β, remplie par une triangulation non structurée (éléments P_2), qui permet de suivre le bord courbe de β et de raffiner le maillage à son voisinage.

Comme Ω est un rectangle aligné sur la grille, son bord est formé d'arêtes de quadrilatères : aucun triangle n'est nécessaire le long de $\partial\Omega$, et la condition $u = 0$ s'y impose directement sur les nœuds de la grille.

3.1 Définition de l'obstacle β

On décrit β en coordonnées polaires autour d'un centre $c = (c_x, c_y)$. Pour $x \in \mathbb{R}^2$, on note $\rho(x) = |x - c|$ et $\theta(x)$ l'angle polaire de $x - c$ (`numpy.arctan2`). On se donne un rayon $r(\theta) > 0$, 2π-périodique et régulier, et on pose

$$\beta = \{x : \rho(x) \le r(\theta(x))\}, \qquad \partial\beta = \{\gamma(t),\ t \in [0, 2\pi)\}, \quad \gamma(t) = c + r(t) \begin{pmatrix} \cos t \\ \sin t \end{pmatrix}.$$

Deux formes sont utilisées :

- **Disque** (validation) : $r(\theta) = R$.
- **Cacahuète** (non convexe) : $r(\theta) = r_0\,(1 + \varepsilon \cos 2\theta)$, avec $0 \le \varepsilon < 1$ pour que $r > 0$. Le bord est concave en $\theta = \pm\pi/2$ dès que $\varepsilon > 1/5$.

Les opérations géométriques dont le code a besoin sont alors :

- **test d'appartenance** : $x \in \beta \iff \rho(x) \le r(\theta(x))$;
- **distance** $d(x, \beta)$: distance minimale de x à un échantillonnage fin de γ (par exemple 2000 points), mise à zéro si $x \in \beta$;
- **fonction de niveau** $\phi_\beta(x) = \rho(x)^2 - r(\theta(x))^2$, négative dans β, nulle sur $\partial\beta$ et positive dans D. Elle est régulière dans D car le centre c est dans β.

Notations de taille. Dans tout le document, h désigne le pas de la grille, h_β le pas sur $\partial\beta$ et δ la largeur de la zone de transition. Pour les études de convergence, on fait varier h en gardant h_β/h et δ/h constants ; les estimations $O(h^k)$ portent sur ce h.

3.2 Construction du maillage

Le maillage se construit ainsi :

1. **Sélection des cellules.** On garde les cellules de la grille dont les quatre sommets et le centre sont hors de β et à distance au moins m_β de β. On retire ensuite, jusqu'à stabilisation :
   - les îlots de cellules qui ne sont pas reliés (par des arêtes) à la rangée de cellules le long de $\partial\Omega$, par exemple des cellules enfermées entre plusieurs composantes de β : ils sont rendus à la zone $\mathcal{T}$, pour que chaque trou de $\mathcal{Q}$ soit bordé par un seul contour (composantes connexes calculées avec `scipy.ndimage.label`) ;
   - les cellules qui n'ont qu'une seule voisine (par une arête) ;
   - les deux cellules d'un sommet qui ne se touchent que par ce coin (sinon ce sommet serait partagé par quatre arêtes du bord de $\mathcal{Q}$).
2. **Contours intérieurs de $\mathcal{Q}$.** Les cellules non gardées forment un ou plusieurs *trous* de $\mathcal{Q}$ (composantes connexes par les arêtes) ; chacun contient au moins une composante de β (section 3.6). Les arêtes de la grille qui appartiennent à une seule cellule de $\mathcal{Q}$ forment $\partial\Omega$ et un contour polygonal fermé autour de chaque trou, qui ne doit pas toucher $\partial\Omega$; une arête est rattachée au trou de la cellule non gardée qu'elle borde. Pour un obstacle connexe, il y a un seul trou. Les nœuds de ces contours sont imposés : ce sont exactement les nœuds de la grille. Chacune de ses arêtes est donc commune à un quadrilatère et à un triangle, et le maillage est conforme : sur cette arête, les traces Q_2 et P_2 sont toutes deux quadratiques et partagent les trois mêmes nœuds (deux sommets et le milieu).
3. **Bord de β.** On le discrétise avec un pas $h_\beta < h$, en répartissant les points selon l'abscisse curviligne, ce qui fonctionne pour un β non convexe. En pratique, on calcule la longueur $s(t)$ de γ sur un échantillonnage fin, on choisit $n_\beta = \lceil s(2\pi)/h_\beta \rceil$ points, et on garde pour chacun son paramètre t_i (obtenu en inversant s).

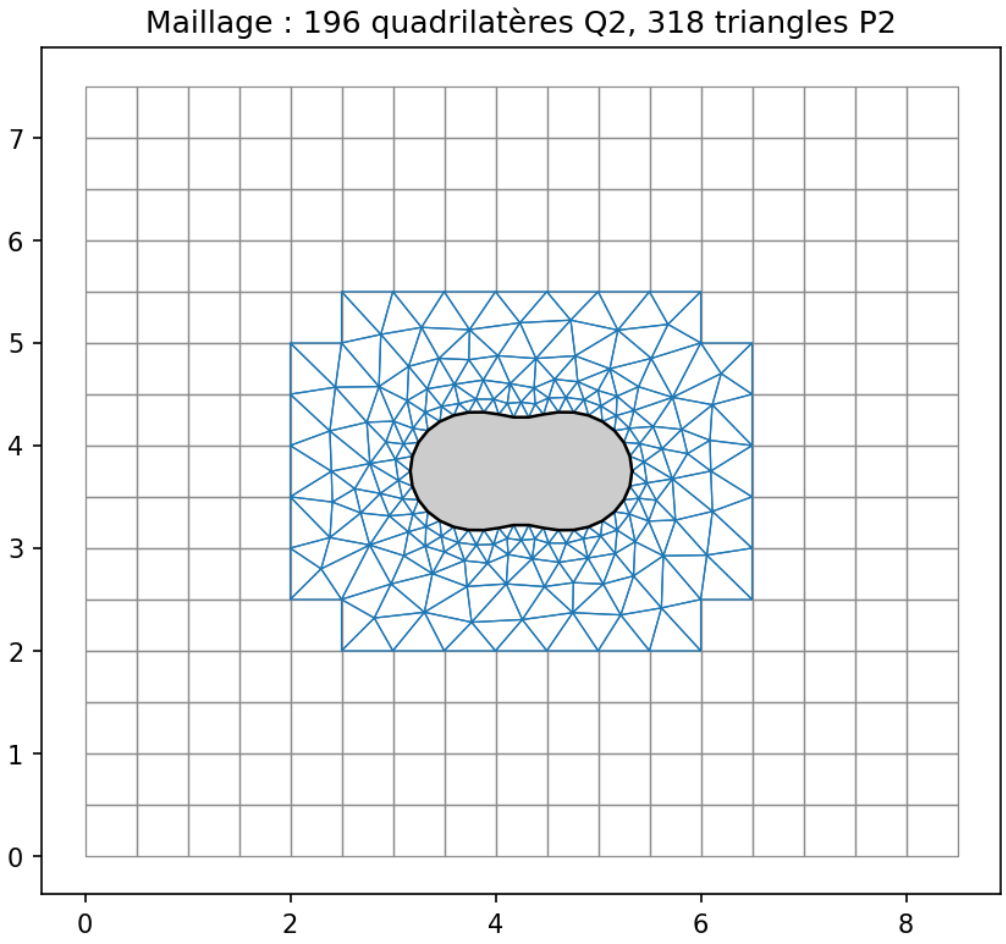
4. **Triangulation de $\mathcal{T}$.** Dans chaque trou de $\mathcal{Q}$, on calcule une triangulation de Delaunay contrainte par son contour et par le bord des composantes de β qu'il contient (repérées par la cellule où tombe un point de leur bord), avec une taille de maille cible

$$h_{\mathrm{loc}}(x) = h_\beta + (h - h_\beta)\,\min\!\left(1, \frac{d(x,\beta)}{\delta}\right),$$

où $d(x,\beta)$ est la distance à β et δ la largeur de la zone de transition. Les mailles sont ainsi fines près de β, où la solution varie le plus, et de taille h au raccord avec la grille. Les trous sont triangulés séparément, par des appels successifs à `gmsh`, et les triangles obtenus sont réunis.

5. **Éléments isoparamétriques.** Pour chaque arête posée sur $\partial\beta$, entre deux points de paramètres t_a et t_b, le nœud milieu P_2 est placé sur $\partial\beta$ en $\gamma(\frac{t_a+t_b}{2})$ (en tenant compte du passage de 2π à 0 pour la dernière arête), et non au milieu du segment. Le triangle devient courbe, décrit par la transformation quadratique de l'élément de référence ; sans cela, l'erreur géométrique limiterait la convergence en norme H^1 à $O(h^{3/2})$ au lieu de $O(h^2)$.

En pratique, on utilise le logiciel `gmsh` depuis Python. On lui donne les deux contours (celui de β et celui de la grille, dont les nœuds sont fixés), puis il remplit la zone entre les deux avec des triangles, en suivant la taille de maille $h_{\mathrm{loc}}(x)$ définie ci-dessus. La figure ci-dessous est produite par `code/main.py` avec les paramètres par défaut de la section 7.1 ($h = 0.5$, $h_\beta = 0.15$, $\delta = 0.8$, $m_\beta = 1$, obstacle en forme de cacahuète), le rectangle $\Omega = [0; 8.5] \times [0; 7.5]$ étant construit autour de β comme à la section 3.5 avec $d_{\min} = m_\beta + 4h = 3$; le maillage compte 196 quadrilatères et 318 triangles. Les arêtes posées sur $\partial\beta$ sont tracées courbes.



3.3 Numérotation globale des nœuds

La grille et `gmsh` numérotent leurs nœuds chacun de leur côté. On construit une numérotation globale unique de la façon suivante :

1. **Sommets de la grille.** Le sommet (ih, jh), $0 \le i \le n_x$, $0 \le j \le n_y$, reçoit un numéro s'il est sommet d'au moins une cellule de $\mathcal{Q}$.
2. **Sommets des triangles.** `gmsh` est appelé avec des éléments d'ordre 1 (triangles à 3 nœuds), une fois par trou de $\mathcal{Q}$. Chaque point des contours est créé dans `gmsh` comme un point géométrique, et `gmsh` renvoie le nœud qui lui correspond : un nœud du contour intérieur de $\mathcal{Q}$ reprend le numéro du sommet de la grille, et un point de $\partial\beta$ reçoit un nouveau numéro en gardant son indice k dans la discrétisation de γ. Les autres nœuds, intérieurs à $\mathcal{T}$, reçoivent un nouveau numéro.
3. **Milieux d'arêtes.** On parcourt toutes les arêtes des triangles et des quadrilatères. Un dictionnaire, dont la clé est le couple trié $(\min(a,b), \max(a,b))$ des numéros des deux sommets, associe à chaque arête le numéro de son milieu. Le milieu n'est créé qu'à la première rencontre de l'arête : une arête commune à deux éléments (en particulier à un quadrilatère et à un triangle) a donc un seul milieu. Si l'arête est sur $\partial\beta$, le milieu est placé sur la courbe (étape 5 de la section 3.2).
4. **Centres des quadrilatères.** Chaque cellule de $\mathcal{Q}$ reçoit un nœud supplémentaire en son centre.
5. **Nœuds de bord.** Un nœud est sur $\partial\Omega$ si $x \in \{0, L_x\}$ ou $y \in \{0, L_y\}$ (à la tolérance près). Il est sur $\partial\beta$ s'il est un point de discrétisation de γ, ou le milieu d'une arête dont les deux sommets sont des points *consécutifs* de cette discrétisation. Une arête qui relie deux points non voisins de $\partial\beta$ est une corde : elle traverse D et son milieu reste un nœud intérieur.

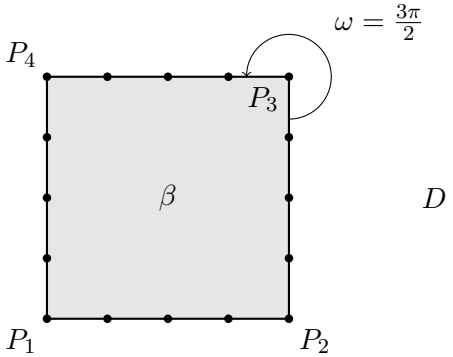
3.4 Cas d'un obstacle polygonal

On considère maintenant un obstacle β polygonal. Le reste de la méthode (grille, zone $\mathcal{Q}$, triangulation de $\mathcal{T}$ par `gmsh`, numérotation) est inchangé ; seules la description de β, la discrétisation de son bord et le traitement des nœuds milieux changent.

Définition. On se donne m sommets $P_1, \dots, P_m$, numérotés dans le sens trigonométrique, et on pose $P_{m+1} = P_1$. Le polygone doit être simple (ses côtés ne se coupent pas), mais il n'a pas besoin d'être convexe ni étoilé. Le côté j est le segment $[P_j, P_{j+1}]$, de longueur $\ell_j = |P_{j+1} - P_j|$. On paramètre $\partial\beta$ par l'abscisse curviligne : avec $s_1 = 0$, $s_{j+1} = s_j + \ell_j$ et $L = s_{m+1}$ le périmètre,

$$\gamma(s) = P_j + \frac{s - s_j}{\ell_j}\,(P_{j+1} - P_j) \qquad \text{pour } s \in [s_j, s_{j+1}].$$

γ est continue mais seulement affine par morceaux : elle n'est pas dérivable aux sommets P_j.



Opérations géométriques. Elles remplacent celles de la section 3.1 :

— **test d'appartenance** : méthode du lancer de rayon. On compte le nombre de côtés coupés par la demi-droite horizontale issue de x ; $x \in \beta$ si ce nombre est impair (`matplotlib.path.Path.contains_points`) ;

— **distance** $d(x, \beta)$: elle est maintenant exacte, sans échantillonnage. Pour chaque côté,

$$d(x, [P_j, P_{j+1}]) = |x - P_j - \tau_j(P_{j+1} - P_j)|,$$

$$\tau_j = \min\left(1, \max\left(0, \frac{(x - P_j) \cdot (P_{j+1} - P_j)}{\ell_j^2}\right)\right),$$

puis $d(x, \beta) = \min_j d(x, [P_j, P_{j+1}])$, mise à zéro si $x \in \beta$.

Discrétisation du bord. On ne répartit plus les points uniformément sur tout le périmètre : les sommets P_j doivent être des points de discrétisation, sinon le maillage « couperait » les coins de β. On découpe donc chaque côté séparément en $n_j = \lceil \ell_j / h_\beta \rceil$ segments égaux :

$$x_{j,i} = P_j + \frac{i}{n_j}(P_{j+1} - P_j), \qquad 0 \le i < n_j.$$

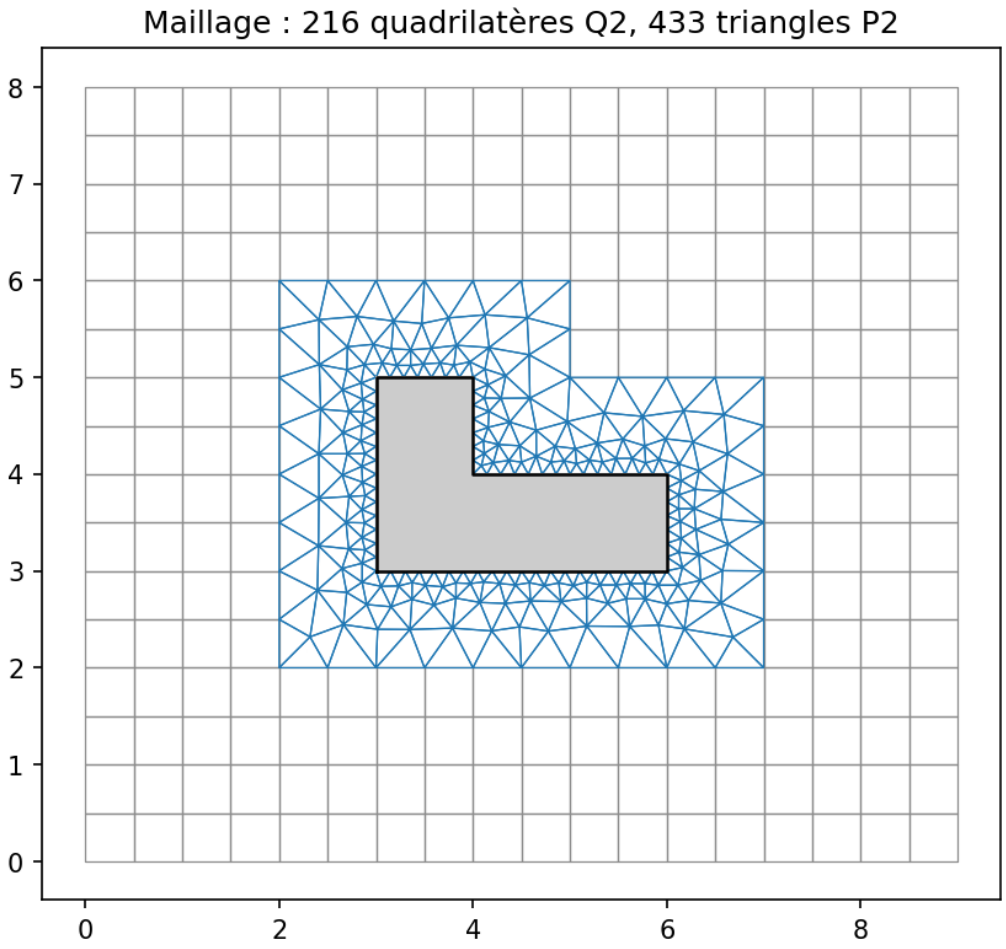
La liste de tous les $x_{j,i}$, dans l'ordre des côtés, donne les $n_\beta = \sum_j n_j$ points de $\partial\beta$, que l'on transmet à `gmsh` comme dans le cas courbe.

Éléments. Chaque arête posée sur $\partial\beta$ relie deux points consécutifs d'un même côté : elle est contenue dans $\partial\beta$. Son nœud milieu est donc simplement le milieu du segment, et l'étape 5 de la section 3.2 (placement sur γ) n'est plus nécessaire. Tous les triangles sont à côtés droits : Φ_T est affine, $|\det D\Phi_T| = 2|T|$ est constant, et la quadrature à 6 points calcule R^K et M^K exactement. Le domaine maillé D_h coïncide exactement avec D : il n'y a plus d'erreur géométrique et $V_h \subset H_0^1(D)$.

Exemple. La figure ci-dessous est produite par `code/main.py` avec $h = 0.5$, $h_\beta = 0.15$, $\delta = 0.8$, $m_\beta = 1$, pour un obstacle en forme de L de sommets

$$(2, 1.5),\ (5, 1.5),\ (5, 2.5),\ (3, 2.5),\ (3, 3.5),\ (2, 3.5).$$

Ce polygone n'est pas convexe : il possède cinq coins saillants ($\alpha_j = \pi/2$, donc $\omega_j = 3\pi/2$ pour D) et un coin rentrant en $(3, 2.5)$ ($\alpha_j = 3\pi/2$). Le domaine Ω est construit autour de β comme à la section 3.5, avec $d_{\min} = m_\beta + 4h = 3$ (qui vérifie bien $d_{\min} \ge m_\beta + h$) : on obtient $\Omega = [0, 9] \times [0, 8]$, l'obstacle étant translaté de $(1, 1.5)$. Le maillage compte 216 quadrilatères et 433 triangles, sans raffinement aux coins. Les sommets du polygone sont bien des nœuds du maillage, et toutes les arêtes posées sur $\partial\beta$ sont droites.



Singularités aux coins. Un sommet P_j où l'angle intérieur de β vaut $\alpha_j < \pi$ (coin saillant de β) est un coin *rentrant* pour D, d'angle $\omega_j = 2\pi - \alpha_j > \pi$. En coordonnées polaires (r, θ) centrées en P_j, la solution se comporte près de ce coin comme

$$u(r,\theta) \sim C\, r^{\lambda_j} \sin(\lambda_j \theta), \qquad \lambda_j = \frac{\pi}{\omega_j} < 1,$$

dont le gradient est infini en $r = 0$. On a seulement $u \in H^{1+\lambda-\varepsilon}(D)$ pour tout $\varepsilon > 0$, avec $\lambda = \min_j \lambda_j$. Sur un maillage à raffinement uniforme, l'ordre 2 des éléments finis ne sert plus à rien près du coin et on observe

$$\|u - u_h\|_{H^1} = O(h^{\lambda}), \qquad \|u - u_h\|_{L^2} = O(h^{2\lambda}).$$

Pour un obstacle carré, $\omega = 3\pi/2$ et $\lambda = 2/3$: on attend $O(h^{2/3})$ en norme H^1 au lieu de $O(h^2)$. Seuls les sommets où $\alpha_j > \pi$ (coins rentrants de β, donc saillants pour D) ne posent pas de problème.

Raffinement aux coins. Pour retrouver l'ordre optimal, on raffine le maillage de façon graduée vers chaque sommet P_j tel que $\alpha_j < \pi$. Avec $r(x) = \min_j |x - P_j|$ (minimum pris sur ces sommets), on remplace la taille cible par

$$h_{\mathrm{loc}}^{\mathrm{coin}}(x) = \max\left(h_{\min},\ \min\left(h_{\mathrm{loc}}(x),\ h_\beta \left(\frac{r(x)}{\delta} \right)^{1-\mu} \right) \right),$$

où h_{loc} est la taille de la section 3.2. Pour des éléments de degré $k = 2$, le paramètre de gradation doit vérifier $\mu < \lambda/k$, soit $\mu < 1/3$ pour un carré (on prend par exemple $\mu = 1/4$). La taille

minimale $h_{\min}$, de l'ordre de $h^{1/\mu}$, évite une taille nulle au sommet. Le nombre de mailles reste de l'ordre de h^{-2} et on retrouve $O(h^2)$ en norme H^1 et $O(h^3)$ en norme L^2. Il suffit de transmettre $h_{\mathrm{loc}}^{\mathrm{coin}}$ à `gmsh` à la place de h_{loc} ; le reste du code est inchangé.

Validation. Pour un obstacle rectangulaire $\beta = [x_0, x_1] \times [y_0, y_1]$, on garde la solution manufacturée de la section 6 en remplaçant g par

$$g(x, y) = X(x)\, Y(y), \qquad X = (x - x_0)(x - x_1), \qquad Y = (y - y_0)(y - y_1),$$

qui s'annule sur $\partial\beta$ (ainsi que sur les prolongements des côtés, ce qui ne gêne pas). On a $\nabla g = (X'Y,\ XY')$ avec $X' = 2x - x_0 - x_1$, $Y' = 2y - y_0 - y_1$, et $\Delta g = 2(X + Y)$, d'où

$$f = (1 + a^2 + b^2)\, S\, g - 2\, \nabla S \cdot \nabla g - S\, \Delta g.$$

Cette solution est régulière : elle ne présente pas de singularité aux coins, et l'on doit observer les ordres optimaux même sans raffinement aux coins. Elle valide donc le code, mais pas le traitement des coins. Pour ce dernier, on résout avec $f = 1$ et on compare à une solution de référence calculée sur un maillage beaucoup plus fin : l'ordre observé doit passer de $\lambda \approx 2/3$ (maillage uniforme) à 2 (maillage gradué) en norme H^1.

3.5 Construction de Ω à partir d'un obstacle polygonal

Plutôt que de fixer Ω à l'avance, on peut le construire autour de β en imposant une distance minimale $d_{\min}$ entre chaque côté de β et $\partial\Omega$.

Réduction à la boîte englobante. Ω est un rectangle aligné sur les axes, contenant β. La distance d'un segment $[P_j, P_{j+1}]$ à la droite $x = X_0$ est une fonction affine le long du segment : elle est atteinte en l'une de ses extrémités. Il en est de même pour les trois autres côtés de Ω, donc

$$\min_j d([P_j, P_{j+1}], \partial\Omega) = \min\left(x_{\min} - X_0,\ X_1 - x_{\max},\ y_{\min} - Y_0,\ Y_1 - y_{\max}\right),$$

où $x_{\min} = \min_j x_{P_j}$, $x_{\max} = \max_j x_{P_j}$ (de même en y) et $\Omega = [X_0, X_1] \times [Y_0, Y_1]$. Imposer la distance $d_{\min}$ à chaque segment revient donc à imposer une marge $d_{\min}$ autour de la boîte englobante $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ des sommets.

Alignement sur la grille. Les dimensions de Ω doivent être des multiples de h. On prend

$$n_x = \left\lceil \frac{x_{\max} - x_{\min} + 2d_{\min}}{h} \right\rceil, \qquad n_y = \left\lceil \frac{y_{\max} - y_{\min} + 2d_{\min}}{h} \right\rceil,$$

$L_x = n_x h$, $L_y = n_y h$, et on centre β :

$$X_0 = \frac{x_{\min} + x_{\max}}{2} - \frac{L_x}{2}, \qquad Y_0 = \frac{y_{\min} + y_{\max}}{2} - \frac{L_y}{2}.$$

La marge obtenue de chaque côté est au moins $d_{\min}$. Pour se ramener à $\Omega = [0, L_x] \times [0, L_y]$, on translate les sommets de β de $-(X_0, Y_0)$; le reste du code est inchangé.

Choix de $d_{\min}$. Le contour intérieur de $\mathcal{Q}$ ne doit pas toucher $\partial\Omega$ (section 3.2). Une cellule le long de $\partial\Omega$, par exemple $[X_0, X_0 + h] \times [y, y + h]$, est à distance au moins $d_{\min} - h$ de β : elle est gardée dans $\mathcal{Q}$ dès que

$$d_{\min} \geq m_\beta + h.$$

Toute la rangée de cellules le long de $\partial\Omega$ est alors dans $\mathcal{Q}$ et sépare $\partial\Omega$ du contour autour de β.

Étude de convergence. Ω ne doit pas dépendre de h, sinon on ne compare pas les solutions d'un même problème. On construit Ω une seule fois, avec le pas le plus grossier $h_{\max}$ et $d_{\min} \geq m_\beta + h_{\max}$; comme les pas suivants sont $h_{\max}/2^k$, L_x et L_y restent des multiples de chacun d'eux.

Remarque. Ω est ici un domaine artificiel : la condition $u = 0$ sur $\partial\Omega$ n'a de sens que si elle est imposée par le problème à cette distance, ou si l'on approche un problème posé sur un domaine non borné, auquel cas $d_{\min}$ doit être grand devant la taille de β.

En Python, la fonction de `geometry.py`

```
domain_from_obstacle(obstacle, h, d_min, m_beta)
```

renvoie l'obstacle translaté, L_x, L_y et la translation appliquée. Elle accepte aussi un obstacle importé d'un fichier CAO (section 3.6) : les sommets P_j sont alors remplacés par les points d'échantillonnage de $\partial\beta$.

3.6 Obstacle importé d'un fichier CAO (STEP)

Plutôt que de décrire β par une formule ou par la liste de ses sommets, on peut le dessiner dans un logiciel de CAO et l'exporter au format STEP (`.step`), le format d'échange standard des modeleurs géométriques. Un tel fichier décrit exactement le bord de β comme une suite de courbes : segments, arcs de cercle ou d'ellipse, B-splines… Le reste de la méthode (grille, zone $\mathcal{Q}$, triangulation de $\mathcal{T}$, numérotation) est inchangé : seule la description de β change, et elle combine le cas courbe (section 3.1) et le cas polygonal (section 3.4).

Lecture du fichier. Le fichier est lu par `gmsh`, avec le noyau géométrique OpenCASCADE. Il doit décrire une géométrie plane, contenue dans un plan $z = \text{cte}$: soit une face, dont on prend le bord, soit directement un contour fermé de courbes. Un volume est refusé. On obtient m courbes $C_1, \ldots, C_m$, chacune donnée par un paramétrage $u \mapsto C_j(u)$, $u \in [u_j^-, u_j^+]$, que `gmsh` sait évaluer exactement. Les coordonnées sont lues telles qu'elles sont écrites dans le fichier ; un facteur d'échelle permet de changer d'unité (les logiciels de CAO travaillent souvent en millimètres).

Mise bout à bout des courbes. Les courbes ne sont pas rangées dans l'ordre du contour dans le fichier, et leurs sens de parcours sont quelconques. On part de C_1, puis on cherche à chaque étape la courbe dont une extrémité coïncide (à une tolérance relative 10^{-6} près) avec l'extrémité finale de la courbe courante, en la retournant si nécessaire, jusqu'à revenir au point de départ. On obtient ainsi un contour fermé $C_1, \ldots, C_m$ où la fin de C_j est le début de C_{j+1}.

Paramétrage de γ. Le paramètre u fourni par la CAO n'est pas l'abscisse curviligne, et il faut évaluer γ après la fermeture de `gmsh`, puisque `gmsh` est de nouveau appelé pour la triangulation de $\mathcal{T}$. On échantillonne donc finement chaque courbe (environ 4000 points au total, répartis selon la longueur des courbes), on calcule l'abscisse curviligne cumulée s le long du contour, de longueur totale L, et on pose $t = 2\pi s/L \in [0, 2\pi)$. Entre deux échantillons, $\gamma(t)$ est obtenu par interpolation linéaire. Si Δs est l'écart entre deux échantillons et κ la courbure, l'écart entre la corde et la courbe est au plus

$$\frac{\kappa\,\Delta s^2}{8},$$

de l'ordre de 10^{-7} pour les exemples ci-dessous, très petit devant h_β. On note t_j le paramètre du début de C_j et ℓ_j la longueur de C_j.

Discrétisation du bord et éléments. Comme pour un polygone, les extrémités des courbes (les coins éventuels de β) doivent être des nœuds. On découpe donc chaque courbe séparément en $n_j = \lceil \ell_j / h_\beta \rceil$ morceaux de même longueur, soit les paramètres

$$t_{j,i} = t_j + \frac{i}{n_j}\,(t_{j+1} - t_j), \qquad 0 \le i < n_j.$$

En revanche, contrairement au cas polygonal, les courbes ne sont pas des segments : le nœud milieu d'une arête posée sur $\partial\beta$ est placé sur la courbe en $\gamma(\frac{t_a+t_b}{2})$, comme à l'étape 5 de la section 3.2. Les deux extrémités d'une telle arête sont sur une même courbe C_j, et son milieu aussi.

Opérations géométriques.

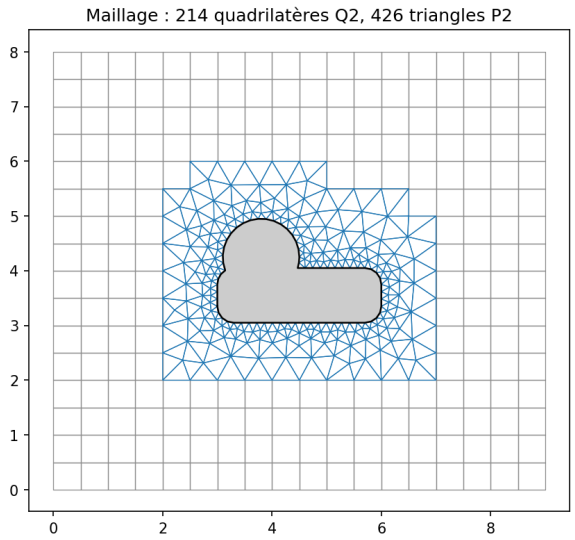
- **test d'appartenance** : lancer de rayon sur le polygone formé par les échantillons (`matplotlib.path.Path.contains_points`) ;
- **distance** $d(x, \beta)$: distance au plus proche échantillon de γ (arbre `scipy.spatial.cKDTree`), mise à zéro si $x \in \beta$;
- **construction de** Ω : celle de la section 3.5, avec la boîte englobante des échantillons.

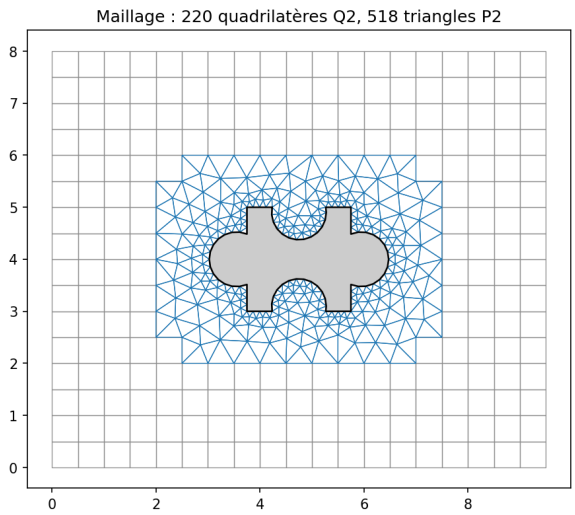
Aux coins de β, les remarques de la section 3.4 sur les singularités et le raffinement restent valables.

Validation. On exporte en STEP le disque de validation ($c = (3.5, 2.5)$, $R = 1$), on le relit avec `from_step` et on reprend l'étude de convergence de la section 6, avec les mêmes paramètres. On retrouve les ordres optimaux, ce qui confirme que les nœuds milieux sont bien placés sur la courbe :

h	erreur L^2	ordre	erreur H^1	ordre
0.5	$8.95 \cdot 10^{-3}$	–	$1.42 \cdot 10^{-1}$	–
0.25	$1.25 \cdot 10^{-3}$	2.84	$3.90 \cdot 10^{-2}$	1.87
0.125	$1.61 \cdot 10^{-4}$	2.96	$9.98 \cdot 10^{-3}$	1.96
0.0625	$2.04 \cdot 10^{-5}$	2.98	$2.52 \cdot 10^{-3}$	1.98

Exemples. Les fichiers STEP sont rangés dans `code/cao/`. `obstacle.step` (rectangle arrondi réuni à un disque, généré par `cao/make_obstacle.py`) mêle segments, arcs et coins rentrants ; `Puzzle.step`, dessiné en CAO, est une pièce de puzzle bordée de 15 courbes. Avec les paramètres par défaut et $d_{\min} = m_\beta + 4h = 3$, `code/main.py` donne respectivement $\Omega = [0, 9] \times [0, 8]$ (214 quadrilatères, 426 triangles) et $\Omega = [0, 9.5] \times [0, 8]$ (220 quadrilatères, 518 triangles). Les coins sont des nœuds du maillage et les arcs sont suivis par des arêtes courbes.





Obstacle non connexe

Le fichier peut contenir plusieurs faces disjointes : β est alors la réunion de composantes connexes $\beta_1, \ldots, \beta_p$, et la mise bout à bout des courbes produit un contour fermé par composante. Chaque composante est décrite comme ci-dessus, avec son propre paramétrage γ_c, $c = 1, \ldots, p$, et

$$d(x, \beta) = \min_c d(x, \beta_c), \qquad x \in \beta \iff \exists\, c,\ x \in \beta_c.$$

La construction du maillage change en deux points :

— **Triangulation.** Chaque trou de $\mathcal{Q}$ contient une ou plusieurs composantes. La partie de $\mathcal{T}$ qui le remplit est limitée à l'extérieur par le contour du trou et possède un trou par composante qu'il contient : on transmet à `gmsh` ce contour et les contours $\partial\beta_c$ correspondants.

— **Numérotation.** Un point de $\partial\beta$ est repéré par le couple (c, k) : composante et indice dans la discrétisation de γ_c. Une arête est posée sur $\partial\beta$ si ses deux sommets appartiennent à la *même* composante et sont consécutifs dans sa discrétisation ; son milieu est alors placé sur γ_c. Une arête qui relie deux composantes traverse D : son milieu est un nœud intérieur.

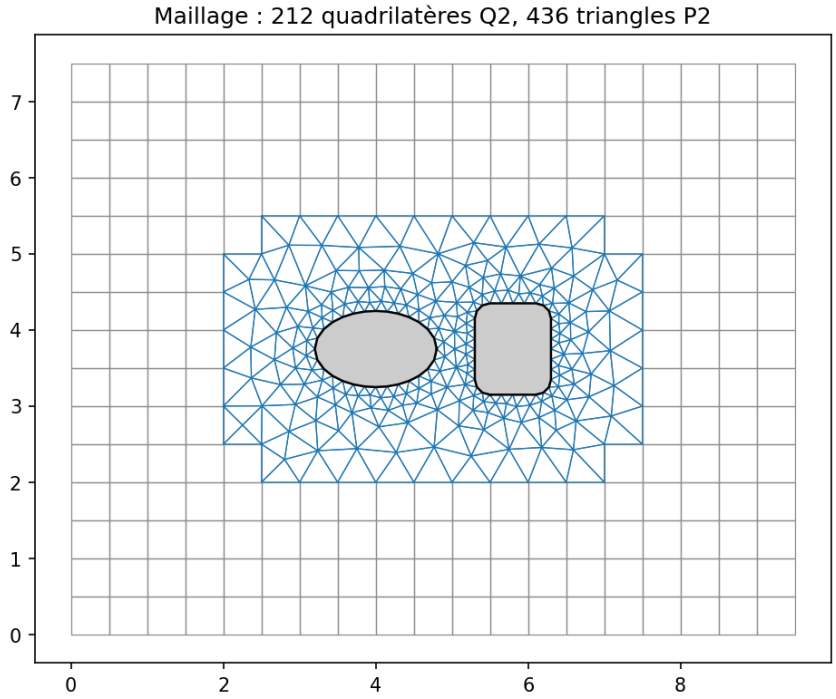
Pour $p = 1$, on retrouve exactement le cas précédent.

Composantes proches ou éloignées. Une cellule située entre deux composantes n'est gardée que si elle est à distance au moins m_β de chacune, ce qui est impossible si leur écart est inférieur à $2m_\beta$ environ. Des composantes *proches* partagent donc un même trou de $\mathcal{Q}$, entouré par une seule zone $\mathcal{T}$. Si elles sont plus éloignées, des quadrilatères passent entre elles et $\mathcal{Q}$ a plusieurs trous, chacun triangulé séparément (étapes 2 et 4 de la section 3.2). Les deux situations peuvent coexister. Si des cellules de $\mathcal{Q}$ sont enfermées entre plusieurs composantes, sans lien avec le bord de Ω, elles sont retirées (étape 1) et le trou englobe tout le groupe. Sont refusés un β troué (une composante contient une autre) et deux faces qui se chevauchent.

Validation et exemple. Pour deux disques $\beta_c = D(c_c, R_c)$, avec $c_1 = (2.5, 2.5)$, $R_1 = 0.7$, $c_2 = (4.6, 2.5)$, $R_2 = 0.6$, on prend la solution manufacturée $u = S\, g_1 g_2$, où $g_c(x) = |x - c_c|^2 - R_c^2$, qui s'annule sur les deux bords, et $f = -\Delta u + u$. L'écart entre les disques vaut 0.8 : on fixe $m_\beta = 0.45$ pour tous les pas afin qu'ils partagent toujours le même trou de $\mathcal{Q}$ (avec $m_\beta = 0.9\, h$, des quadrilatères passent entre eux dès $h = 0.25$ et la topologie du maillage change). On obtient encore les ordres optimaux :

h	erreur L^2	ordre	erreur H^1	ordre
0.5	$2.42 \cdot 10^{-1}$	–	3.80	–
0.25	$3.11 \cdot 10^{-2}$	2.96	$9.66 \cdot 10^{-1}$	1.98
0.125	$3.90 \cdot 10^{-3}$	2.99	$2.42 \cdot 10^{-1}$	2.00
0.0625	$4.88 \cdot 10^{-4}$	3.00	$6.05 \cdot 10^{-2}$	2.00

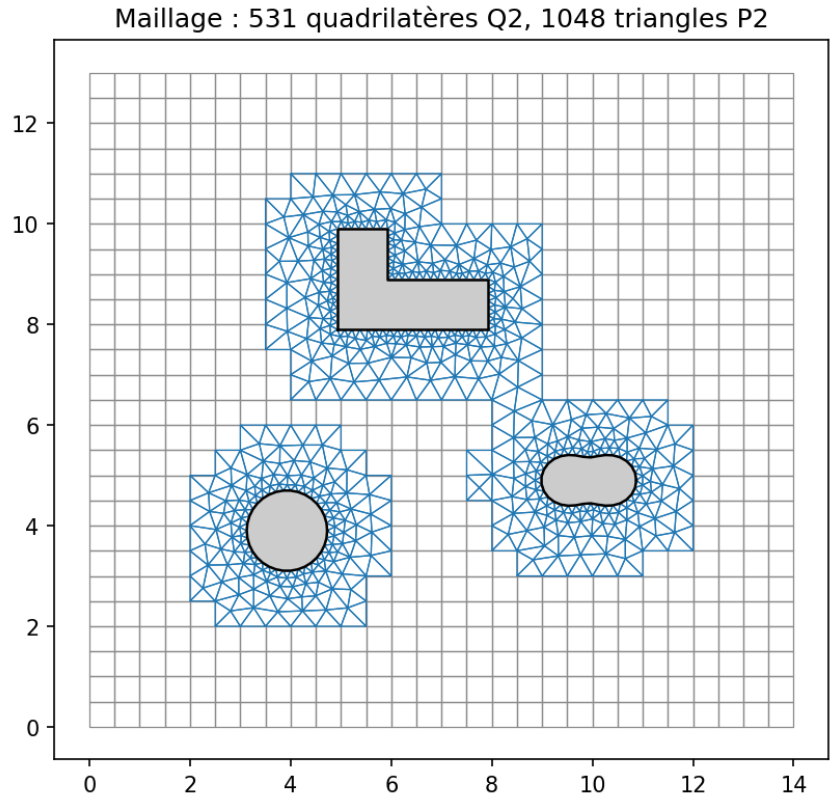
Le fichier `cao/deux_obstacles.step` (une ellipse et un rectangle arrondi) donne, avec les paramètres par défaut, $\Omega = [0, 9.5] \times [0, 7.5]$, 212 quadrilatères et 436 triangles :



Pour des composantes éloignées, `code/main.py` réunit (classe `MultiObstacle`) un disque de centre $(0,0)$ et de rayon 0.8, une cacahuète de centre $(6,1)$ ($r_0 = 0.7$, $\varepsilon = 0.35$) et le polygone en L de sommets

$$(1,4),\ (1,6),\ (2,6),\ (2,5),\ (4,5),\ (4,4).$$

Avec les paramètres par défaut, on obtient $\Omega = [0,14]\times[0,13]$, 531 quadrilatères et 1048 triangles. Des quadrilatères passent entre le disque et les deux autres composantes : $\mathcal{Q}$ a deux trous, l'un autour du disque, l'autre autour de la cacahuète et du L, assez proches pour partager la même zone $\mathcal{T}$.



En Python, `from_step(path, scale)` de `geometry.py` renvoie un objet `CurveObstacle` (une composante) ou `MultiObstacle` (plusieurs composantes), qui se passe directement à `domain_from_obstacl` puis à `build_mesh`. Pour produire un fichier STEP depuis Python, on construit la géométrie avec `gmsh.model.occ` (`addDisk`, `addRectangle`, `addBSpline`, opérations booléennes…) puis on l'écrit avec `gmsh.write("obstacle.step")`.

4 Discrétisation

4.1 Espaces polynomiaux et espace V_h

On définit les espaces de polynômes P_k (utilisés sur les triangles) et Q_k (utilisés sur les quadrilatères) :

$$P_k = \left\{ p(x,y) = \sum_{m=0}^{k} \sum_{n=0}^{k-m} \alpha_{m,n}\, x^m y^n \right\},$$

$$Q_k = \left\{ p(x,y) = \sum_{m=0}^{k} \sum_{n=0}^{k} \alpha_{m,n}\, x^m y^n \right\}.$$

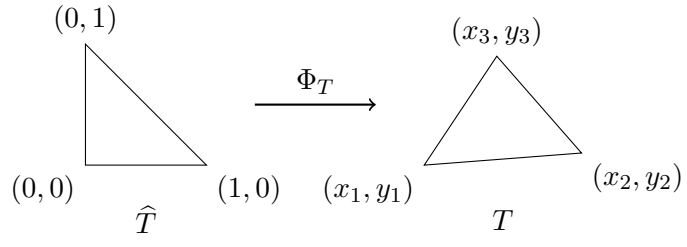
On note $\mathcal{T}_h$ l'ensemble des triangles, $\mathcal{Q}_h$ l'ensemble des quadrilatères et D_h le domaine qu'ils recouvrent. Avec les transformations Φ_T et Φ_Q vers les éléments de référence (section 4.2), on pose

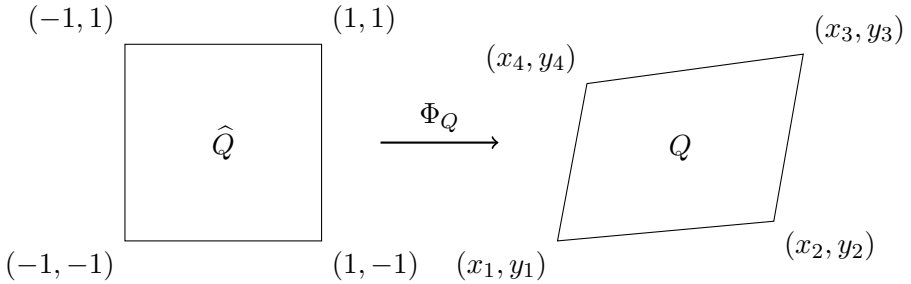
$$V_h = \Big\{ v \in C^0(\overline{D_h}) \ : \ v|_T \circ \Phi_T \in P_2 \ \ \forall T \in \mathcal{T}_h, \ \ v|_Q \circ \Phi_Q \in Q_2 \ \ \forall Q \in \mathcal{Q}_h, \ \ v = 0 \text{ sur } \partial D_h \Big\}.$$

Les fonctions de V_h sont continues et polynomiales par morceaux, donc dans $H^1(D_h)$. La continuité est assurée par le maillage : sur chaque arête commune à deux éléments, les deux restrictions sont des polynômes de degré 2 de l'abscisse le long de l'arête, qui coïncident en trois points (les deux sommets et le milieu), donc partout. V_h est engendré par les fonctions de base nodales φ_j (valant 1 au nœud j et 0 aux autres nœuds) associées aux nœuds qui ne sont pas sur ∂D_h. L'espace W_h de la formulation variationnelle est défini de la même façon, sans la condition $v = 0$ sur ∂D_h : il est engendré par les φ_j de tous les nœuds.

4.2 Passage à l'élément de référence

Chaque élément K du maillage est l'image d'un élément de référence $\widehat{K}$ par une transformation Φ_K : le triangle $\widehat{T}$ de sommets $(0,0)$, $(1,0)$, $(0,1)$ pour les triangles, le carré $\widehat{Q} = [-1,1]^2$ pour les quadrilatères.





On décrit la géométrie avec les mêmes fonctions de base que la solution (éléments *isoparamétriques*). Si K a n_K nœuds x_k ($n_K = 6$ pour un triangle, $n_K = 9$ pour un quadrilatère) et si $\widehat{\psi}_k$ sont les fonctions P_2 ou Q_2 de la section 5.2 :

$$\Phi_K(\hat{x}) = \sum_{k=1}^{n_K} x_k \, \widehat{\psi}_k(\hat{x}), \qquad D\Phi_K(\hat{x}) = \sum_{k=1}^{n_K} x_k \, \nabla\widehat{\psi}_k(\hat{x})^\top \quad \text{(matrice } 2 \times 2\text{)}.$$

Pour toute fonction φ définie sur K, on note $\widehat{\varphi} = \varphi \circ \Phi_K$. La dérivation des fonctions composées et le changement de variables donnent

$$\nabla\varphi \circ \Phi_K = D\Phi_K^{-\top} \, \nabla\widehat{\varphi}, \qquad \int_K g \, \mathrm{d}x = \int_{\widehat{K}} g \circ \Phi_K \, |\det D\Phi_K| \, \mathrm{d}\hat{x}.$$

En particulier, si le nœud global i est le nœud local k de K, alors $\varphi_i \circ \Phi_K = \widehat{\psi}_k$: toutes les intégrales se calculent avec les fonctions de référence.

Cas particuliers.

— **Triangle à côtés droits** (milieux au milieu des arêtes) : Φ_T est affine,

$$\Phi_T(\hat{x}) = x_1 + \left[\, x_2 - x_1 \;\middle|\; x_3 - x_1 \,\right] \hat{x}, \qquad |\det D\Phi_T| = 2\,|T|.$$

— **Carré de la grille** de côté h et de centre x_c :

$$\Phi_Q(\hat{x}) = x_c + \tfrac{h}{2}\,\hat{x}, \qquad D\Phi_Q = \tfrac{h}{2} I, \qquad |\det D\Phi_Q| = \tfrac{h^2}{4}.$$

— **Triangle courbe** (une arête sur $\partial\beta$, milieu placé sur la courbe) : Φ_T est quadratique et $D\Phi_T$ dépend du point. C'est ce qui permet à l'élément de suivre le bord de β.

5 Système linéaire et structure des matrices

5.1 Du problème discret au système linéaire

On note N le nombre total de nœuds du maillage (sommets, milieux d'arêtes et centres des quadrilatères), y compris les nœuds de bord, et $(\varphi_j)_{1 \le j \le N}$ les fonctions nodales associées, avec $\varphi_j = 1$ au nœud j et 0 aux autres. Seules celles des nœuds intérieurs forment une base de V_h ; on travaille d'abord avec toutes, et les nœuds de bord sont éliminés à la section 5.6. On cherche

$$u_h = \sum_{j=1}^{N} U_j \, \varphi_j, \qquad U = (U_1, \ldots, U_N)^\top \in \mathbb{R}^N.$$

En prenant $v_h = \varphi_i$ dans (P_h) pour $i = 1, \ldots, N$, on obtient le système linéaire

$$\boxed{A\,U = F, \qquad A = R + M}$$

avec

$$R_{ij} = \int_D \nabla\varphi_j \cdot \nabla\varphi_i, \qquad M_{ij} = \int_D \varphi_j \, \varphi_i, \qquad F_i = \int_D f \, \varphi_i.$$

R est la matrice de rigidité, M la matrice de masse et F le second membre. On a $A_{ij} = a(\varphi_j, \varphi_i)$: A est donc symétrique définie positive.

5.2 Fonctions de base P_2 et Q_2 sur les éléments de référence

Les fonctions de degré 2 $\widehat{\psi}_k$ servent à la fois à approcher u et à décrire la géométrie des éléments (section 4.2).

Triangle P_2 (6 nœuds). Sur $\widehat{T}$, avec $\lambda_1 = 1-\hat{x}-\hat{y}$, $\lambda_2 = \hat{x}$, $\lambda_3 = \hat{y}$, les nœuds sont numérotés : sommets 1:$(0,0)$, 2:$(1,0)$, 3:$(0,1)$, puis milieux 4:$(\frac{1}{2},0)$, 5:$(\frac{1}{2},\frac{1}{2})$, 6:$(0,\frac{1}{2})$.

$$\begin{aligned}
&\widehat{\psi}_1 = \lambda_1(2\lambda_1-1), && \nabla\widehat{\psi}_1 = (4\lambda_1-1)\begin{pmatrix}-1\\-1\end{pmatrix},\\
&\widehat{\psi}_2 = \hat{x}(2\hat{x}-1), && \nabla\widehat{\psi}_2 = \begin{pmatrix}4\hat{x}-1\\0\end{pmatrix},\\
&\widehat{\psi}_3 = \hat{y}(2\hat{y}-1), && \nabla\widehat{\psi}_3 = \begin{pmatrix}0\\4\hat{y}-1\end{pmatrix},\\
&\widehat{\psi}_4 = 4\lambda_1\hat{x}, && \nabla\widehat{\psi}_4 = 4\begin{pmatrix}1-2\hat{x}-\hat{y}\\-\hat{x}\end{pmatrix},\\
&\widehat{\psi}_5 = 4\hat{x}\hat{y}, && \nabla\widehat{\psi}_5 = 4\begin{pmatrix}\hat{y}\\\hat{x}\end{pmatrix},\\
&\widehat{\psi}_6 = 4\lambda_1\hat{y}, && \nabla\widehat{\psi}_6 = 4\begin{pmatrix}-\hat{y}\\1-\hat{x}-2\hat{y}\end{pmatrix}.
\end{aligned}$$

Quadrilatère Q_2 (9 nœuds). Sur $\widehat{Q} = [-1,1]^2$, on utilise les polynômes de Lagrange 1D de degré 2 aux points $-1, 0, 1$:

$$\ell_{-1}(s) = \tfrac{1}{2}s(s-1), \qquad \ell_0(s) = 1-s^2, \qquad \ell_1(s) = \tfrac{1}{2}s(s+1),$$

$$\ell'_{-1}(s) = s-\tfrac{1}{2}, \qquad \ell'_0(s) = -2s, \qquad \ell'_1(s) = s+\tfrac{1}{2}.$$

Les nœuds sont numérotés : sommets 1:$(-1,-1)$, 2:$(1,-1)$, 3:$(1,1)$, 4:$(-1,1)$, milieux 5:$(0,-1)$, 6:$(1,0)$, 7:$(0,1)$, 8:$(-1,0)$, et centre 9:$(0,0)$. Si le nœud k a pour coordonnées (a_k, b_k), alors

$$\widehat{\psi}_k(\hat{x},\hat{y}) = \ell_{a_k}(\hat{x})\,\ell_{b_k}(\hat{y}), \qquad \nabla\widehat{\psi}_k = \begin{pmatrix}\ell'_{a_k}(\hat{x})\,\ell_{b_k}(\hat{y})\\ \ell_{a_k}(\hat{x})\,\ell'_{b_k}(\hat{y})\end{pmatrix}.$$

5.3 Quadrature

Les intégrales sur l'élément de référence sont approchées par

$$\int_{\widehat{K}} g(\hat{x})\,\mathrm{d}\hat{x} \approx \sum_{q=1}^{n_q} w_q\, g(\hat{x}_q).$$

— **Triangle** : règle de Gauss à 6 points, exacte pour les polynômes de degré 4. Avec $a = 0.445948490915965$ et $b = 0.091576213509771$, les points sont

$$(a,a),\ (1-2a,a),\ (a,1-2a) \quad \text{de poids } w_a = 0.111690794839005,$$

$$(b,b),\ (1-2b,b),\ (b,1-2b) \quad \text{de poids } w_b = 0.054975871827661.$$

La somme des poids vaut $\frac{1}{2}$, l'aire de $\widehat{T}$.

— **Quadrilatère** : Gauss–Legendre 3×3. Les points sont les (s_m, s_n) et les poids les produits $w_m w_n$, avec

$$s \in \{-\sqrt{3/5},\ 0,\ \sqrt{3/5}\}, \qquad w \in \{\tfrac{5}{9},\ \tfrac{8}{9},\ \tfrac{5}{9}\}.$$

Les valeurs $\widehat{\psi}_k(\hat{x}_q)$ et $\nabla\widehat{\psi}_k(\hat{x}_q)$ ne dépendent pas de l'élément : on les calcule une seule fois.

5.4 Matrices élémentaires

Pour un élément K à n_K nœuds ($n_K = 6$ pour un triangle, $n_K = 9$ pour un quadrilatère), de transformation Φ_K, on note en chaque point de quadrature $\hat{x}_q$:

— $J_q = D\Phi_K(\hat{x}_q)$ et $d_q = |\det J_q|$;
— $\Psi_q = \big(\widehat{\psi}_1(\hat{x}_q), \dots, \widehat{\psi}_{n_K}(\hat{x}_q)\big)^\top \in \mathbb{R}^{n_K}$;
— $G_q = \big[\nabla\widehat{\psi}_1(\hat{x}_q) \ \cdots \ \nabla\widehat{\psi}_{n_K}(\hat{x}_q)\big] \in \mathbb{R}^{2\times n_K}$;
— $B_q = J_q^{-\top} G_q \in \mathbb{R}^{2\times n_K}$, qui contient les gradients des fonctions de base sur l'élément réel.

Les matrices et le vecteur élémentaires s'écrivent alors

$$R^K = \sum_q w_q\, d_q\, B_q^\top B_q, \qquad M^K = \sum_q w_q\, d_q\, \Psi_q \Psi_q^\top, \qquad F^K = \sum_q w_q\, d_q\, f\big(\Phi_K(\hat{x}_q)\big)\, \Psi_q,$$

avec $R^K, M^K \in \mathbb{R}^{n_K\times n_K}$ symétriques et $F^K \in \mathbb{R}^{n_K}$. On pose $A^K = R^K + M^K$.

Cas des carrés de la grille. Pour une cellule de côté h, $J_q = \frac{h}{2} I$ et $d_q = \frac{h^2}{4}$ en tout point. On obtient

$$R^K = \widehat{R}, \qquad M^K = \frac{h^2}{4}\,\widehat{M},$$

où $\widehat{R}$ et $\widehat{M}$ sont calculées sur $\widehat{Q}$ avec $d_q = 1$ et $B_q = G_q$. Elles sont identiques pour tous les quadrilatères : il suffit de les calculer une seule fois.

5.5 Assemblage

Chaque élément K possède une table de connectivité $\mathrm{conn}_K(k)$ qui donne le numéro global du nœud local k. Un nœud partagé par plusieurs éléments (sommet ou milieu d'arête) a un seul numéro global. L'assemblage consiste à ajouter chaque contribution élémentaire à la bonne place :

1. Initialiser $A = 0 \in \mathbb{R}^{N\times N}$ et $F = 0 \in \mathbb{R}^N$.
2. Pour chaque élément K (triangles puis quadrilatères) :
   (a) calculer A^K et F^K ;
   (b) pour $k, l = 1, \dots, n_K$: $A_{\mathrm{conn}_K(k),\,\mathrm{conn}_K(l)}$ += A^K_{kl} ;
   (c) pour $k = 1, \dots, n_K$: $F_{\mathrm{conn}_K(k)}$ += F^K_k.

Structure de A. $A_{ij} \neq 0$ seulement si les nœuds i et j appartiennent à un même élément. A est donc *creuse* : chaque ligne contient au plus quelques dizaines de termes non nuls (25 pour un sommet commun à quatre quadrilatères Q_2), quel que soit N. En Python, on stocke les triplets (i, j, A^K_{kl}) dans trois listes, puis on construit la matrice au format `scipy.sparse.coo_matrix`, qui additionne automatiquement les doublons, avant de la convertir au format CSR pour la résolution.

5.6 Conditions aux limites et système final

On sépare les nœuds en deux ensembles :

— $\mathcal{I}$: les nœuds intérieurs ;
— $\mathcal{B}$: les nœuds situés sur $\partial\Omega$ ou sur $\partial\beta$ (y compris les milieux P_2 placés sur la courbe $\partial\beta$). On note $\mathcal{B}_\beta \subset \mathcal{B}$ ceux de $\partial\beta$.

En renumérotant les inconnues dans cet ordre, le système s'écrit par blocs

$$\begin{pmatrix} A_{\mathcal{I}\mathcal{I}} & A_{\mathcal{I}\mathcal{B}} \\ A_{\mathcal{B}\mathcal{I}} & A_{\mathcal{B}\mathcal{B}} \end{pmatrix} \begin{pmatrix} U_{\mathcal{I}} \\ U_{\mathcal{B}} \end{pmatrix} = \begin{pmatrix} F_{\mathcal{I}} \\ F_{\mathcal{B}} \end{pmatrix}.$$

Les conditions au bord fixent $U_{\mathcal{B}} = G_{\mathcal{B}}$, avec

$$G_j = 0 \ \text{ pour } j \in \mathcal{B} \setminus \mathcal{B}_\beta \ (\text{nœuds de } \partial\Omega), \qquad G_j = g_\beta(x_j) \ \text{ pour } j \in \mathcal{B}_\beta.$$

Les milieux P_2 des arêtes de $\partial\beta$ étant placés exactement sur la courbe, g_β y est évaluée sans erreur géométrique. Les lignes associées à $\mathcal{B}$ ne sont plus des équations à résoudre (on ne teste qu'avec les φ_i, $i \in \mathcal{I}$, qui forment une base de V_h). La première ligne de blocs donne alors

$$\boxed{A_{\mathcal{I}\mathcal{I}}\, U_{\mathcal{I}} = F_{\mathcal{I}} - A_{\mathcal{I}\mathcal{B}}\, G_{\mathcal{B}}}$$

Le terme $-A_{\mathcal{I}\mathcal{B}}G_{\mathcal{B}}$ est la version discrète de $-a(u_g, v)$: il transporte la condition au bord dans le second membre. $A_{\mathcal{I}\mathcal{I}}$ est symétrique définie positive, donc le système admet une unique solution. On le résout avec `scipy.sparse.linalg.spsolve` (ou un gradient conjugué), puis on reconstruit U en complétant par $U_{\mathcal{B}} = G_{\mathcal{B}}$. La matrice A et le vecteur F sont assemblés comme dans le cas homogène : seul le second membre du système réduit change. Si $g_\beta = 0$, on retrouve $A_{\mathcal{I}\mathcal{I}}U_{\mathcal{I}} = F_{\mathcal{I}}$.

5.7 Récapitulatif des tableaux

Nom	Taille	Contenu
`coords`	$N \times 2$	coordonnées (x, y) de chaque nœud
`tri`	$n_T \times 6$	numéros globaux des nœuds de chaque triangle
`quad`	$n_Q \times 9$	numéros globaux des nœuds de chaque quadrilatère
`bnd`	$\lvert\mathcal{B}\rvert$	indices des nœuds de $\partial\Omega \cup \partial\beta$
`bnd_beta`	$\lvert\mathcal{B}_\beta\rvert$	indices des nœuds de $\partial\beta$ (sous-ensemble de `bnd`)
A^K, F^K	$n_K \times n_K$, n_K	matrice et vecteur élémentaires
A	$N \times N$ (creuse)	$R + M$ assemblée
F	N	second membre assemblé
U	N	valeurs de u_h aux nœuds

L'ordre des nœuds locaux dans `tri` et `quad` doit être le même que celui des éléments de référence ci-dessus.

6 Validation

6.1 Solution manufacturée

Pour vérifier le code, on choisit une solution exacte u_{ex} qui s'annule sur $\partial\Omega$ mais pas sur $\partial\beta$, afin de tester aussi la condition non homogène. On en déduit f, et on prend $g_\beta = u_{\text{ex}}$ (restreinte à $\partial\beta$). On prend $\Omega = [0, L_x] \times [0, L_y]$ et pour β le disque de centre c et de rayon R. Avec $a = \pi/L_x$ et $b = \pi/L_y$:

$$S(x, y) = \sin(ax)\sin(by), \qquad \phi(x, y) = (x - c_x)^2 + (y - c_y)^2 - R^2, \qquad u_{\text{ex}} = S\,(\phi + 1).$$

S s'annule sur $\partial\Omega$ et ϕ sur $\partial\beta$, donc $u_{\text{ex}} = 0$ sur $\partial\Omega$ et $u_{\text{ex}} = S$ sur $\partial\beta$; S n'est pas nulle sur $\partial\beta$ puisque β est à l'intérieur de Ω. On a

$$\nabla u_{\text{ex}} = (\phi + 1)\,\nabla S + S\,\nabla\phi, \qquad \nabla S = \begin{pmatrix} a\cos(ax)\sin(by) \\ b\sin(ax)\cos(by) \end{pmatrix}, \qquad \nabla\phi = 2\begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix},$$

et, avec $\Delta S = -(a^2 + b^2)S$ et $\Delta\phi = 4$,

$$f = -\Delta u_{\text{ex}} + u_{\text{ex}} = (1 + a^2 + b^2)\,S\,(\phi + 1) - 4\big[a(x - c_x)\cos(ax)\sin(by) + b(y - c_y)\sin(ax)\cos(by)\big] - 4S.$$

6.2 Calcul des erreurs

Les erreurs sont calculées élément par élément avec la même quadrature que pour l'assemblage. Avec les notations de la section sur les matrices élémentaires, et $U^K = (U_{\mathrm{conn}_K(k)})_k$ les valeurs de u_h aux nœuds de l'élément K :

$$\|u_{\mathrm{ex}} - u_h\|_{L^2}^2 \approx \sum_K \sum_q w_q \, d_q \Big(u_{\mathrm{ex}}(x_q) - \Psi_q^\top U^K \Big)^2,$$

$$|u_{\mathrm{ex}} - u_h|_{H^1}^2 \approx \sum_K \sum_q w_q \, d_q \Big| \nabla u_{\mathrm{ex}}(x_q) - B_q \, U^K \Big|^2,$$

où $x_q = \Phi_K(\hat{x}_q)$. On a $\|u_{\mathrm{ex}} - u_h\|_{H^1}^2 = \|u_{\mathrm{ex}} - u_h\|_{L^2}^2 + |u_{\mathrm{ex}} - u_h|_{H^1}^2$.

6.3 Étude de convergence

On résout le problème pour $h \in \{0.5,\ 0.25,\ 0.125,\ 0.0625\}$, en gardant $h_\beta = 0.3\,h$, $\delta = 1.6\,h$ et $m_\beta = 0.9\,h$. Pour deux pas successifs h et $h/2$, l'ordre observé, pour une erreur $E(h)$, est

$$p = \frac{\log\left(E(h)/E(h/2)\right)}{\log 2}.$$

On doit observer $p \approx 3$ pour l'erreur L^2 et $p \approx 2$ pour l'erreur H^1. Un ordre H^1 proche de 1.5 indique en général que les milieux des arêtes de $\partial\beta$ ne sont pas placés sur la courbe. Un ordre proche de 1 ou une erreur qui ne diminue pas indique une erreur dans les fonctions de base, la numérotation locale ou l'assemblage. Une erreur qui stagne dès le premier pas indique souvent que g_β n'est pas imposée (ou mal évaluée) aux nœuds de $\partial\beta$.

6.4 Cas d'application

Une fois le code validé, on le lance sur l'obstacle en forme de cacahuète avec $f = 1$. Il n'y a pas de solution exacte : on vérifie seulement que u_h est positive, nulle sur ∂D et régulière. On fait de même pour le disque, l'obstacle polygonal (section 3.4), les obstacles lus dans les fichiers STEP de `code/cao/` (section 3.6), y compris l'obstacle non connexe, et les trois obstacles éloignés de la section 3.6 (avec $g_\beta = 1 + x/L_x$ sur $\partial\beta$).

Pour illustrer la condition non homogène, on reprend la cacahuète avec $f = 1$ et $g_\beta(x, y) = 1 + x/L_x$ sur $\partial\beta$: u_h doit valoir g_β sur $\partial\beta$, s'annuler sur $\partial\Omega$ et décroître régulièrement entre les deux.

7 Paramètres et sorties attendues

7.1 Paramètres par défaut

Paramètre	Valeur	Rôle
L_x, L_y	7, 5	Ω de validation (L_x/h, L_y/h entiers)
$d_{\min}$	$m_\beta + 4h = 3$	marge entre β et $\partial\Omega$ (applications)
h	0.5	pas de la grille
h_β	0.15	pas sur $\partial\beta$
δ	0.8	largeur de la zone de transition
m_β	1	distance minimale entre $\mathcal{Q}$ et β
disque	$c = (3.5, 2.5)$, $R = 1$	obstacle de validation
cacahuète	$c = (3.1, 2.63)$, $r_0 = 0.8$,	obstacle d'application
	$\varepsilon = 0.35$	(puis translatée avec Ω)

7.2 Organisation du code

Module	Contenu
`geometry.py`	$r(\theta)$, $\gamma(t)$, test $x \in \beta$, $d(x,\beta)$, ϕ_β, polygone, lecture STEP (`from_step`), construction de Ω
`mesh.py`	sélection de $\mathcal{Q}$, appel à `gmsh`, numérotation, nœuds P2/Q2
`reference.py`	$\widehat{\psi}_k$, $\nabla\widehat{\psi}_k$, points et poids de quadrature
`assembly.py`	Φ_K, $D\Phi_K$, R^K, M^K, F^K, assemblage creux
`solver.py`	élimination des nœuds de $\mathcal{B}$ ($U_\mathcal{B} = G_\mathcal{B}$), résolution
`main.py`	cas de validation, étude de convergence, cas d'application, ligne de commande, menu, validation du maillage, figures
`cao/`	fichiers STEP d'exemple et script `make_obstacle.py`

Dépendances : `numpy`, `scipy`, `gmsh`, `matplotlib`.

7.3 Utilisation de `main.py`

Validation du maillage. Pour un obstacle donné, le programme construit Ω autour de β (section 3.5, avec $d_{\min} = m_\beta + 4h$), puis le maillage, affiche le nombre de nœuds, de triangles et de quadrilatères, et montre le maillage dans une fenêtre avec deux boutons « Valider » et « Refuser ». L'assemblage et la résolution (pour $f = 1$) ne sont lancés qu'après validation. En cas de refus ou d'échec du maillage, on peut modifier les paramètres h, h_β, δ, m_β (Entrée conserve la valeur courante), changer d'obstacle ou quitter. Sans affichage disponible (*backend* `matplotlib` non interactif), le maillage est écrit dans `resultats/` et la validation se fait au clavier.

Ligne de commande.

Commande	Effet
`python3 main.py`	menu (voir ci-dessous)
`python3 main.py` *cas*	cas prédéfini : `disque`, `cacahuete`, `cacahuete_dirichlet`, `polygone`, `step`, `puzzle`, `non_connexe`, `eloignes`
`python3 main.py -step f.step`	obstacle lu dans un fichier STEP / IGES / BREP (`-echelle`)
`python3 main.py -obstacle ...`	β composé, option à répéter pour chaque composante
`python3 main.py validation`	étude de convergence (section 6)
`python3 main.py tous`	validation et tous les cas prédéfinis en parallèle, sans validation du maillage

Les options `-h`, `-h-beta`, `-delta` et `-m-beta` changent les paramètres de maillage, et `-y` (`-oui`) valide le maillage sans l'afficher. Dans le mode `tous` et pour l'étude de convergence, chaque calcul tourne dans son propre processus (`ProcessPoolExecutor`), sans fenêtre.

Obstacle composé. β peut être construit comme réunion de composantes dont on choisit le type et les paramètres :

Composante	Paramètres
`disque`	centre (x,y), rayon R
`cacahuete`	centre (x,y), r_0, ε
`polygone`	translation (x,y), puis sommets $x_1\ y_1\ x_2\ y_2 \ldots$ (au moins trois ; L par défaut)
`fichier`	chemin d'un fichier CAO, translation (x,y), échelle

Une composante lue dans un fichier peut elle-même être non connexe : toutes ses composantes sont ajoutées. Par exemple

```
python3 main.py -obstacle disque 0 0 1 -obstacle polygone 4 0
```

réunit un disque de rayon 1 et le polygone en L translaté de $(4, 0)$. Une seule composante donne un obstacle connexe, plusieurs un `MultiObstacle` (section 3.6), traité comme à la section 3.2 que les composantes soient proches ou éloignées.

Menu. Sans argument, le programme propose : `composer` (obstacles ajoutés un par un), `validation` ou `tous`. En mode `composer`, on choisit le type puis les paramètres de chaque composante (Entrée : valeur par défaut ; les composantes sont placées côte à côte par défaut, $x = 4k$ pour la k-ième). Après chaque ajout ou retrait, le maillage est recalculé et affiché sans bloquer (ou écrit dans `maillage_compose_`p`.png`, p étant le nombre de composantes) ; une composante qui fait échouer le maillage n'est pas ajoutée. On termine avec `t`, puis le maillage final passe par la validation décrite ci-dessus.

7.4 Sorties

- `maillage.png` : les quadrilatères, les triangles et β pour la cacahuète. Les autres cas ajoutent un suffixe : `_disque`, `_dirichlet` (cacahuète avec $u = g_\beta$ sur $\partial\beta$), `_polygone`, `_step`, `_puzzle`, `_non_connexe`, `_eloignes`, `_compose` (obstacle composé) et le nom du fichier pour `-step` ; de même pour `solution*.png`.
- `solution.png` : u_h en couleurs. Pour l'affichage, chaque triangle P_2 est découpé en 4 sous-triangles et chaque carré Q_2 en 8 à l'aide des nœuds milieux, puis on trace le résultat avec `matplotlib.pyplot.tripcolor`. Une triangulation de Delaunay de tous les nœuds ne convient pas : elle traverserait β.
- `convergence.png` : erreurs L^2 et H^1 en fonction de h en échelle log–log, avec des droites de pente 3 et 2.
- Dans le terminal : le nombre de nœuds, de triangles et de quadrilatères, puis un tableau h, erreur L^2, ordre, erreur H^1, ordre.